

COMPLETANDO O CRUD

Editar e excluir

Como um formulário HTML consegue enviar PUT e DELETE



PUT
DELETE

Projeto `app_escola` · módulo de empréstimos · `method-override`

Desenvolvimento Web — Eixo Informação e Comunicação

E.E.E.P. Lúcia Helena Viana Ribeiro — Horizonte/CE

Prof. Francisco Ericson Cornélio da Costa

2

Onde estamos: as quatro operações

Todo sistema de cadastro faz as mesmas quatro coisas. Duas já estão prontas.

C

Cadastrar

Create

POST

Pronto

R

Listar

Read

GET

Pronto

U

Editar

Update

PUT

Hoje

D

Excluir

Delete

DELETE

Hoje

No módulo de empréstimos, essas quatro palavras têm outro nome

Cadastrar é emprestar. Listar é ver quem está com o quê. Editar é corrigir uma data lançada errada. E excluir, aqui, significa devolver o livro. Guarde essa última: ela muda o que a rota precisa fazer.

3

O problema: o navegador só sabe dois verbos

Um formulário HTML consegue enviar GET e POST. Só isso.

O que gostaríamos de escrever

```
<form action="/emprestimos/3"
      method="DELETE">
  <button>Devolver</button>
</form>

<!-- o navegador ignora o DELETE
      e envia um GET comum -->
```

O que realmente funciona

```
<form action="/emprestimos/3?_method=DELETE"
      method="POST">
  <button>Devolver</button>
</form>

<!-- sai como POST e o Express
      converte para DELETE -->
```

Por que essa limitação existe

PUT e DELETE fazem parte do protocolo HTTP desde sempre, e o JavaScript consegue enviá-los. A restrição está apenas no formulário HTML, que nunca foi atualizado para aceitar outros métodos.

A solução: method-override

Um middleware que lê o parâmetro `_method` da URL e troca o verbo da requisição antes de ela chegar às rotas. Já está instalado no nosso projeto desde a primeira aula.

4

Como o method-override funciona

Ele age entre o navegador e as suas rotas, sem que você precise fazer nada além de configurá-lo.

Navegador

POST /emprestimos/3
?_method=DELETE

O formulário envia POST,
porque é o que ele sabe.

method-override

lê o `_method`

Encontra o parâmetro e troca
o verbo da requisição.

Express

DELETE /emprestimos/3

A rota `router.delete`
é acionada normalmente.

src/app.js – já está lá desde a primeira aula

```
middlewares() {  
  this.app.use(express.urlencoded({ extended: true }));  
  this.app.use(methodOverride("_method")); // <= este  
}
```

5

As rotas que vamos criar

Duas operações, três rotas: editar precisa de uma rota para abrir o formulário e outra para salvar.

`src/routes/emprestimos.js`

```
import { Router } from "express";
import emprestimoController from
"../controllers/emprestimoController.js";

const router = Router();

router.get("/", emprestimoController.index);
router.post("/", emprestimoController.cadastrarEmprestimo);

// NOVAS
router.get("/:id/editar", emprestimoController.editar);
router.put("/:id", emprestimoController.atualizar);
router.delete("/:id", emprestimoController.devolver);

export default router;
```

O `:id` é um parâmetro

Os dois pontos indicam que aquele trecho da URL é variável. No controller, o valor chega em `req.params.id`.

Por que duas rotas para editar

A primeira mostra o formulário preenchido. A segunda recebe os dados alterados e grava.

Mesmo endereço, verbos diferentes

`/emprestimos/3` com GET, PUT ou DELETE são três coisas distintas. É assim que a web foi projetada.

6

Antes de tudo: encontrar o registro

Editar e excluir começam do mesmo jeito, e é aqui que mora o erro mais comum.

find – devolve o objeto

```
const emprestimo = emprestimos.find(
  (e) => e.id_emprestimo === Number(id)
);

// serve para LER e ALTERAR campos
emprestimo.dataDevolucao = novaData;
```

findIndex – devolve a posição

```
const i = emprestimos.findIndex(
  (e) => e.id_emprestimo === Number(id)
);

// serve para REMOVER do array
emprestimos.splice(i, 1);
```

O erro que vai acontecer com metade da turma

req.params.id chega como TEXTO. Comparar "3" === 3 com três iguais dá falso, e o registro nunca é encontrado. A página não quebra: ela simplesmente não faz nada.

Solução: sempre `Number(id)` ou `parseInt(id)` antes de comparar.

É o mesmo problema de req.body

Tudo que chega pela URL ou por formulário é texto. Já vimos isso com a nota do aluno. Agora se repete com o id.

7

Editar, passo 1: abrir o formulário preenchido

A rota GET `/:id/editar` encontra o registro e manda para uma view própria.

`controllers/emprestimoController.js`

```
editar(req, res) {
  const { id } = req.params;

  const emprestimo = emprestimos.find(
    (e) => e.id_emprestimo === Number(id)
  );

  if (!emprestimo) {
    return res.redirect("/emprestimos");
  }

  return res.render("editarEmprestimo", {
    emprestimo,
    livros,
    alunos,
  });
}
```

Por que enviar livros e alunos

O formulário de edição tem os mesmos campos de seleção do cadastro. Sem as listas, os menus chegam vazios.

O if de segurança

Se alguém digitar `/emprestimos/999` na barra de endereço, o `find` devolve `undefined`. Sem esse `if`, a aplicação quebra com erro 500.

8

Editar, passo 2: a view com os valores atuais

Um formulário de edição é o formulário de cadastro com dois acréscimos.

`views/editarEmprestimo.handlebars`

```
<form action="/emprestimos/{{emprestimo.id_emprestimo}}?_method=PUT"
      method="POST">

  <select name="livroId">
    {{#each livros}}
      <option value="{{this.id_livro}}"
        {{#if this.selecionado}}selected{{/if}}>
        {{this.titulo}}
      </option>
    {{/each}}
  </select>

  <input type="date" name="dataDevolucao"
    value="{{emprestimo.dataDevolucao}}" />

  <button>Salvar alterações</button>
</form>
```

1. O `_method=PUT`

Vai na action, junto do id. Sem ele, o Express trata como POST e nenhuma rota responde.

2. Os valores atuais

O atributo `value` nos campos de texto e o `selected` nas opções fazem o formulário abrir já preenchido.

Quem calcula o selecionado?

O controller, como sempre.

9

Editar, passo 3: gravar a alteração

Não se cria um objeto novo. Alteram-se os campos do objeto que já existe.

`controllers/emprestimoController.js`

```
atualizar(req, res) {  
  const { id } = req.params;  
  const { dataEmprestimo, dataDevolucao } = req.body;  
  
  const emprestimo = emprestimos.find(  
    (e) => e.id_emprestimo === Number(id)  
  );  
  
  if (!emprestimo) {  
    return res.redirect("/emprestimos");  
  }  
  
  emprestimo.dataEmprestimo = dataEmprestimo;  
  emprestimo.dataDevolucao = dataDevolucao;  
  
  return res.redirect("/emprestimos");  
}
```

find devolve a referência

O objeto retornado pelo find é o mesmo que está dentro do array. Alterar um campo dele altera o array. Não é preciso reinserir nada.

Sempre redirecionar

Depois de gravar, redirecione para a listagem. Assim o usuário vê o resultado e evita reenviar o formulário ao atualizar a página.

10

Excluir: encontrar a posição e remover

A operação técnica é simples. O que ela significa no sistema é que exige cuidado.

A view: um formulário com um botão só

```
<form action="/emprestimos/{{this.id_emprestimo}}?_method=DELETE"
      method="POST">
  <button>Devolver</button>
</form>
```

O controller: versão ainda incompleta

```
devolver(req, res) {
  const { id } = req.params;

  const i = emprestimos.findIndex(
    (e) => e.id_emprestimo === Number(id)
  );

  if (i === -1) {
    return res.redirect("/emprestimos");
  }

  emprestimos.splice(i, 1);
  return res.redirect("/emprestimos");
}
```

splice(i, 1)

Remove 1 elemento a partir da posição i. É o método que altera o próprio array, diferente do filter, que devolve um array novo.

findIndex devolve -1

Quando não encontra, o retorno é -1 e não undefined. Por isso o if compara com -1.

Mas falta alguma coisa

Este código funciona e ainda assim está errado no nosso sistema. Próximo slide.

11

O que faltava: devolver é liberar o livro

Uma operação em uma entidade pode ter efeito em outra.

Só removendo do array

O empréstimo some da lista, mas o livro continua com `disponivel = false`. Ele nunca mais aparece no menu de novo empréstimo. O acervo vai secando a cada devolução.

Liberando o livro antes de remover

O livro volta a ficar disponível e reaparece no menu. O sistema passa a refletir a realidade da biblioteca.

```
const emprestimo = emprestimos[i];  
  
emprestimo.livro.disponivel = true; // <= a linha que faltava  
  
emprestimos.splice(i, 1);
```

Isso só funciona porque guardamos o objeto livro, e não apenas o id.

`emprestimo.livro` é o mesmo objeto que está dentro do array `livros`. Alterar por um caminho altera pelo outro.

O método devolver, completo

Encontrar, validar, liberar o livro, remover, redirecionar. Nessa ordem.

`controllers/emprestimoController.js`

```
devolver(req, res) {  
  const { id } = req.params;  
  
  const i = empréstimos.findIndex(  
    (e) => e.id_emprestimo === Number(id)  
  );  
  
  if (i === -1) {  
    return res.redirect("/emprestimos");  
  }  
  
  // libera o livro ANTES de remover o empréstimo  
  empréstimos[i].livro.disponivel = true;  
  
  empréstimos.splice(i, 1);  
  
  return res.redirect("/emprestimos");  
}
```

A ordem importa

Depois do splice, o empréstimo já saiu do array e você perde a referência ao livro.

Um comentário útil

Explique o porquê, não o quê. “libera o livro antes de remover” vale mais que “remove do array”.

13

Quatro cuidados que valem nota

Nenhum deles é difícil, e todos aparecem na correção do projeto final.

1

Converter o id

`req.params.id` é texto. Compare sempre com `Number(id)`.

2

Tratar o não encontrado

`find` devolve `undefined`, `findIndex` devolve `-1`. Sempre teste antes de usar.

3

Redirecionar após gravar

Nunca deixe o usuário parado na URL do POST. Sempre `res.redirect`.

4

Confirmar antes de excluir

Um `onclick="return confirm('Confirma?')"` no botão evita exclusão acidental.

O sistema tem que continuar funcionando quando o usuário faz o que você não previu.

Erros comuns e o que verificar

Cinco mensagens que vão aparecer no terminal de vocês nesta semana.

Cannot POST /emprestimos/3

Faltou o `?_method=PUT` na action, ou o formulário não é `method="POST"`.

O botão recarrega e nada muda

O id não foi convertido: `"3" === 3` é falso e o registro não é encontrado.

Cannot read properties of undefined

O registro não existe e faltou o `if` de validação, ou o `splice` veio antes.

O livro nunca mais aparece no menu

A devolução não liberou o livro: falta `disponivel = true`.

O formulário de edição abre vazio

O controller não enviou o objeto, ou a view usa outro nome de variável.

Leia sempre a primeira linha do erro e o nome do arquivo citado nela.

method-override

?_method=PUT na action, method POST.

Rotas

GET /:id/editar, PUT /:id, DELETE /:id.

req.params.id

Chega como texto. Converta sempre.

find

Devolve a referência: altere e pronto.

findIndex + splice

Devolve a posição: use para remover.

Efeito colateral

Devolver libera o livro antes do splice.

Atividade da aula

- Implemente a devolução: botão na tabela, rota DELETE e liberação do livro.
- Implemente a edição das duas datas do empréstimo, com o formulário já preenchido.
- Teste o que acontece ao digitar /emprestimos/999 na barra de endereço e corrija.

Na próxima aula: o projeto final em grupo, onde vocês farão tudo isso sozinhos com outro tema.